Intelligent Wi-Fi Relay Drone for Autonomous Disaster Communication.

Final Project Report — Autonomous Drones (BCSE428L)

A self-deploying chain of relay drones with reinforcement-learned navigation and landing, developed and evaluated in simulation

NAME: LOKESHWARAN P

REG: 23BAI1098

BCSE428L – Autonomous Drones

Code, trained models and results: github.com/cosmosapien200/communication-drone-relay-sim

Table of Contents

1. Abstract **3**
2. Introduction **3**
3. Problem Statement **4**
4. Literature Review **5**
4.1 Drone-based emergency communication (from DA-1) 5
4.2 Quadrotor modelling and control 6
4.3 Reinforcement learning for drone control 6
4.4 RL algorithms and training techniques 6
4.5 Classical navigation and exploration 6
4.6 Simulation platforms 7
4.7 Research gap 7
5. Simulation Environment and Data **7**
5.1 Procedural collapsed towns and curriculum 7
5.2 Radio model 8
5.3 Wind and disturbances 9
5.4 Training and evaluation data 9
6. Methodology **10**
6.1 System architecture 10
6.2 Quadrotor model and flight controller 10
6.3 Navigation as a Markov decision process 11
6.4 Landing as a Markov decision process 12
6.5 Proximal Policy Optimization 13
6.6 Curriculum and robustness to model error 14
6.7 Mission planner 14
6.8 Perception interface (summary) 15
6.9 Baselines 15
7. Implementation **16**
7.1 Code organisation 16
7.2 Training infrastructure 16
7.3 Training runs and reward iteration 17
7.4 Deployment inside the mission and live demonstration 18
8. Experimental Results **19**
8.1 Flight-controller validation 19
8.2 Navigation 20
8.3 Landing 21
8.4 Mission-level results 22
8.5 Effect of perception quality on the mission 24
8.6 Live-demonstration reliability 24
9. Performance Evaluation **25**
9.1 Comparison with the DA-1 proposal 25
9.2 Strengths 25
9.3 Limitations 25
9.4 Failure analysis of the live demonstration 26
10. Conclusion **26**
11. Future Scope **27**
12. References **27**

1. Abstract

Disasters destroy the communication infrastructure that rescue teams depend on. This project develops an autonomous relay-drone system that restores connectivity without any surviving infrastructure: drones are launched one after another from a base station, each one flies through a collapsed town toward a disconnected disaster camp, lands on a safe surface that still has a usable radio link to the previous relay, and becomes the next node of a relay chain. Perching instead of hovering removes the main endurance limit of multirotor relays, because a landed drone only powers its radio.

Because of the limited project time, the complete system was built and validated in simulation. A Unity 6 simulator generates seeded collapsed towns with four difficulty levels, a log-distance radio model with wall losses, mean wind with random gusts, and two flight models behind one velocity-set-point interface: a 1.2 kg rigid-body quadrotor with a cascaded PID controller and rotor mixer, and a fast surrogate matched to it for training. Flight autonomy is split into a rule-based mission planner (frontier exploration, radio-aware landing-spot selection, shared map) and two policies trained with Proximal Policy Optimization (PPO) in Unity ML-Agents: a navigation policy that flies to planner waypoints through unknown streets using only a 24-ray range ring, and a landing policy that descends onto a camera-detected patch under gusts using the output of a U-Net landing-spot detector.

On held-out towns the navigation policy reaches its waypoint in 98 %, 98 %, 93 % and 80 % of episodes on the four difficulty levels, against 93/92/78/54 % for a potential-field controller with the same sensors. The landing policy, trained with a gust curriculum and the noise measured from the real detector, lands in 100 % of episodes at gusts of 0–6 m/s with 1–5 cm error, while a PID visual-servoing baseline falls from 97 % to 44 %. In full multi-drone missions, replacing the learned navigation with the scripted controller cuts the share of missions that reach the camp from 47 % to 9 %. With the real detector in the loop the final system covers the camp in 45 % of 100 held-out level-2 missions and extends coverage by 62.3 m on average, with every link above −80 dBm.

2. Introduction

Floods, earthquakes, cyclones and landslides routinely damage cellular towers, power lines and fibre. Rescue teams lose contact with command centres, victims cannot report their location, and coordination slows down exactly when it matters most. Unmanned aerial vehicles (UAVs) can carry a radio to where it is needed within minutes, which makes them a natural tool for emergency communication.

Two practical problems limit a single relay drone. First, multirotor endurance is short (typically 15–30 minutes of hover). Second, in a dense, partially collapsed town, radio signals are blocked by buildings, so a single relay between a base station and a camp several streets away is usually not enough. This project therefore uses a chain of drones that perch: each drone flies out from the end of the existing chain, explores toward the camp while staying in radio contact with its parent, lands on a safe surface and starts relaying. The next drone receives a shared map and continues from there.

Making this work autonomously is a drone-autonomy problem at several levels: low-level flight control under wind, collision-free navigation through streets that have never been seen, precise landing on a small patch under gusts with an imperfect detector, and mission-level decisions about where to explore and where to land so that the chain keeps making progress without breaking a link. The DA-1 proposal suggested a single PPO-controlled drone that repositions itself using depth features and Wi-Fi metrics. The final work keeps PPO and the communication-driven objective, but extends it to a multi-drone chain, separates mission decisions from learned flight skills, and moves the entire development into simulation, which allowed millions of training steps and evaluation on hundreds of unseen towns within the project time.

- **Contribution 1 – Drone simulator:** procedural collapsed towns with a curriculum of four difficulty levels, radio propagation with wall losses, wind with gusts, and a validated quadrotor flight controller with a matched fast surrogate for RL.
- **Contribution 2 – Learned navigation:** a PPO policy that flies to waypoints through unknown streets from a range-sensor ring alone, trained with a path-progress reward and a difficulty curriculum, and compared with classical baselines.
- **Contribution 3 – Learned landing:** a PPO landing policy trained with potential-based reward shaping, a gust curriculum and the measured noise of the vision detector, compared with PID visual servoing.
- **Contribution 4 – Multi-drone relay mission:** a radio-aware mission planner with frontier exploration, landing-spot selection and a shared map, evaluated end to end on 100 held-out towns per condition, with ablations of every component.

3. Problem Statement

Given a disaster area in which the base station has no usable link to the camp, deploy a sequence of at most six drones that autonomously extend coverage toward the camp by landing at suitable spots, such that every link of the chain stays above the minimum usable signal strength ($T_{link} = -80$ dBm) and the camp is finally covered.

For each drone this decomposes into four autonomy sub-problems:

- **Flight control:** track velocity set-points (≤ 3 m/s horizontal, ≤ 1 m/s vertical, ≤ 90°/s yaw rate) on a quadrotor in mean wind and gusts.
- **Navigation:** reach a planner waypoint 4–80 m away along streets at a fixed cruise altitude of 5 m (below the rooftops), without a map, using only onboard range sensing and the drone's own state; avoid all collisions.
- **Landing:** descend from 3–6 m onto a detected patch of 1.8–5 m side, touching down inside it at ≤ 0.6 m/s, under gusts of up to 4.5 m/s and with the false, missed and displaced detections of a real camera-based detector.
- **Mission decisions:** choose where to explore and where to land so that the chain advances toward the camp while each new relay still receives ≥ −71 dBm from its parent (T_link + 6 dB margin + 3 dB ground penalty).

Constraints: no real flight hardware or flight time was available, so everything had to be trained and validated in simulation; the flight interface had to match what a real companion computer sends to an autopilot (velocity set-points over MAVLink), so that the learned policies remain transferable; and all results had to be measured on towns never used for training.

4. Literature Review

4.1 Drone-based emergency communication (from DA-1)

FSO-based drone-assisted mobile access networks use drones as temporary aerial base stations connected to surviving infrastructure through free-space optical links and optimise placement, user association and bandwidth. They deploy quickly but depend on surviving infrastructure [1].

Multi-UAV wireless mesh networks form a self-organising Wi-Fi mesh for disaster recovery with real-time video and target tracking, demonstrating low latency in field tests, but rely on 5G and non-terrestrial network infrastructure, which raises cost and complexity [2].

Blockchain-enhanced UAV networks secure multi-agency coordination with smart contracts and hybrid consensus, gaining resilience at the price of significant computational and energy overhead [3].

Mozaffari et al. [4] survey UAVs as flying base stations and relays and identify placement, endurance and air-to-ground channel modelling as the key open problems; Al-Hourani et al. [5] show that the optimal altitude of an aerial base station is a trade-off between path loss and line-of-sight probability. In all these works drone positions are computed centrally and the drones are assumed to hover; none addresses how a drone

autonomously reaches and perches at a relay position in a cluttered, unmapped environment.

4.2 Quadrotor modelling and control

A quadrotor is under-actuated: horizontal motion is produced by tilting the total thrust vector. The standard control architecture is therefore cascaded — an outer position or velocity loop computes a desired acceleration, which is converted into a desired attitude and thrust, followed by attitude and body-rate loops and a mixer that maps thrust and torques to the four rotors [6]. Open autopilots such as PX4 [7] implement exactly this cascade and accept velocity set-points from a companion computer in offboard mode, which is the interface our learned policies use.

4.3 Reinforcement learning for drone control

Deep reinforcement learning has been applied at every level of UAV control. Hwangbo et al. [8] trained a neural network policy that controls a quadrotor directly from state and recovers from aggressive throws; Koch et al. [9] compared PPO, TRPO and DDPG for attitude control and found PPO the most reliable, outperforming a tuned PID in simulation. Kaufmann et al. [10] showed that a PPO policy trained in simulation can beat human world champions in real drone racing, demonstrating that simulation-trained policies can transfer to hardware when the dynamics are modelled carefully. For landing, Polvara et al. [11] used deep Q-networks to land a UAV on a marker from camera images, and Rodriguez-Ramos et al. [12] used DDPG to land on a moving platform in simulation and real flight.

4.4 RL algorithms and training techniques

DA-1 compared DQN [13], PPO [14] and SAC [15]. DQN is limited to discrete actions; SAC is sample-efficient but off-policy with a replay buffer and more hyper-parameters; PPO optimises a clipped surrogate objective with generalised advantage estimation [16] and is stable with many parallel environments, which suits a fast simulator. Three training techniques are central here: potential-based reward shaping, which adds guidance without changing the optimal policy [17]; curriculum learning, which presents tasks in increasing difficulty [18]; and randomisation of dynamics and disturbances to obtain policies that are robust to model error [19].

4.5 Classical navigation and exploration

A* search [20] gives optimal paths when the map is known; artificial potential fields [21] provide reactive obstacle avoidance from local sensing but suffer from local minima in concave obstacles such as dead-end streets. Frontier-based exploration [22] drives a robot toward the boundary between known and unknown space and is combined here

with radio constraints; occupancy grids [23] are the basis of our shared map. These methods serve both as components of our planner and as baselines for the learned navigation policy.

4.6 Simulation platforms

DA-1 proposed AirSim [24] and Gazebo with PX4 software-in-the-loop. Unity ML-Agents [25] was used instead because it combines the rendering needed for the camera, a scriptable world generator and a mature PPO trainer, and can run dozens of lightweight training arenas in one headless process. The radio channel follows the log-distance path-loss model [26] with material losses in the range reported by ITU-R P.2040 [27].

4.7 Research gap

Relay-drone research optimises where relays should be but assumes they can get there and hover; RL-for-drones research learns individual skills (attitude control, racing, landing) but not a communication mission. No work combines (a) learned navigation through unknown cluttered streets, (b) learned landing under gusts with a real detector's errors, and (c) a radio-aware multi-drone mission in which every relay perches. This project addresses that gap in simulation and measures how much each learned component contributes at mission level.

5. Simulation Environment and Data

DA-1 listed KITTI, NYU Depth V2 and simulator-generated data. For reinforcement learning the training data is the agent's own experience, so the simulator is the dataset: every training episode and every evaluation run is generated from a seeded procedural town. Training uses even seeds and every evaluation uses odd seeds, so all reported results are on towns never seen during training.

5.1 Procedural collapsed towns and curriculum

Each town is a 96–120 m square on a 1 m grid: an irregular street grid, box buildings 8–20 m tall (the drone cruises at 5 m, below the rooftops, so streets form a maze), collapsed buildings and rubble barriers that create dead ends and detours, landable surfaces (plaza tiles, flat concrete, intact road) and non-landable look-alikes (debris, puddles, holes, wrecked vehicles, slopes). The base station is on one edge and the camp on the far side, usually without line of sight. The same grid drives collisions, range sensors and radio; a 3-D scene with procedural textures is built from it for the cameras.

Table 1. Curriculum levels of the town generator

Parameter	Level 1	Level 2	Level 3	Level 4
Map size (m)	110	96	110	120

Parameter	Level 1	Level 2	Level 3	Level 4
Streets closed by rubble	0 %	8 %	16 %	25 %
Collapsed buildings	0 %	5 %	10 %	15 %
Landable patches / look-alikes (per ha)	40 / 30	35 / 40	25 / 50	14 / 60
Base–camp line of sight	yes	no	no	no
Navigation waypoint distance (m)	4–15	8–30	15–50	20–80
Mean wind / gust amplitude (m/s)	0 / 0	0.3 / 0.3	0.6 / 0.8	1.0 / 1.5

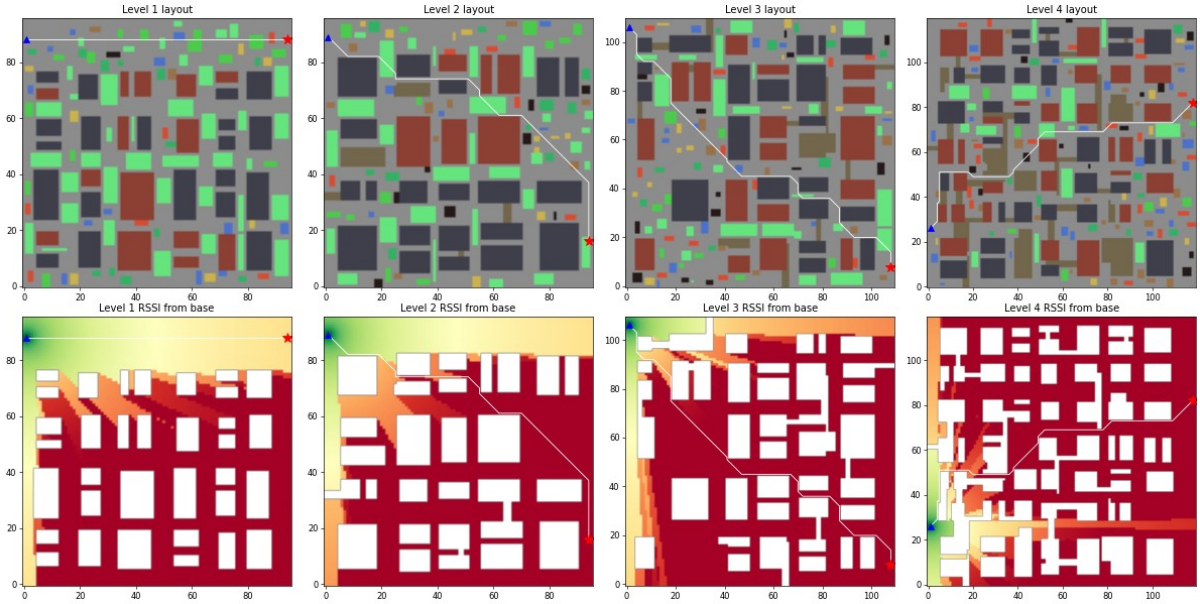


Figure 1. Generated towns for the four levels (top: layout with base ▲, camp ★ and shortest street path; green = landable surfaces) and the radio signal strength from the base (bottom: yellow strong, dark red below the link threshold). Signal follows the streets and dies around corners.

5.2 Radio model

Table 2. Radio propagation model and derived thresholds

Quantity	Value
RSSI	P0 − 10·n·log10(d) − Σ wall losses − ground penalty + noise
P0 / path-loss exponent n	−40 dBm at 1 m / 2.0 (street canyon)
Wall loss per wall crossed	concrete 10 dB, brick 7 dB, rubble 5 dB (two walls

Quantity	Value
	per building)
Ground penalty / noise	3 dB for a landed antenna / σ = 2 dB per reading
Link threshold T_link / margin M	−80 dBm / 6 dB
Exploration limit	frontiers with predicted RSSI ≥ T_link + M = −74 dBm
Landing-spot qualification	measured RSSI ≥ T_link + M + ground = −71 dBm (≈ 35 m line of sight)
Camp covered	last relay → camp ≥ −80 dBm

5.3 Wind and disturbances

Each episode draws a constant mean wind in a random horizontal direction plus a gust process: raised-cosine pulses of 0.8–3 s duration in random directions (with a small vertical component) and peak 40–100 % of the gust amplitude, separated by random gaps of 1.5–5.5 s. Gusts act on the full quadrotor as an external force through aerodynamic drag; in the fast surrogate the high-frequency part of the wind that the velocity loop cannot reject moves the drone with a gain matched to the full model.

5.4 Training and evaluation data

Table 3. Data used for training and evaluation

Data	Source	Size
Navigation experience	32 arenas per Unity process, random start/waypoint pairs on even-seed towns	10.75 M steps (≈ 35 min)
Landing experience	random patch (1.8–5 m), start 3–6 m high and up to 2 m off-centre, gust curriculum	4.3 M + 3.7 M steps (≈ 45 min)
Detector noise for landing	landing-spot locator measured inside the simulator on held-out towns	FN, FP, σ, bias, dropout
Vision dataset (summary)	synthetic, auto-labelled images from 125 towns + targeted fine-tuning sets	13,750 + 9,120 images
Evaluation	odd-seed towns: 100 episodes per level / gust, 100 missions per mission condition	> 4,000 missions

6. Methodology

6.1 System architecture

The system is hierarchical. A rule-based mission planner decides where each drone explores and where it lands; two learned policies execute the flying; a simulated autopilot turns velocity commands into motion; and the vision module supplies the landable-surface map and the landing-spot observations. This split is deliberate: decisions about radio constraints are easy to state as rules and to verify, while reacting to unknown obstacles, gusts and noisy detections is where hand-written controllers are weakest and learning helps most. Everything runs in decision steps of 0.1 s (10 Hz).

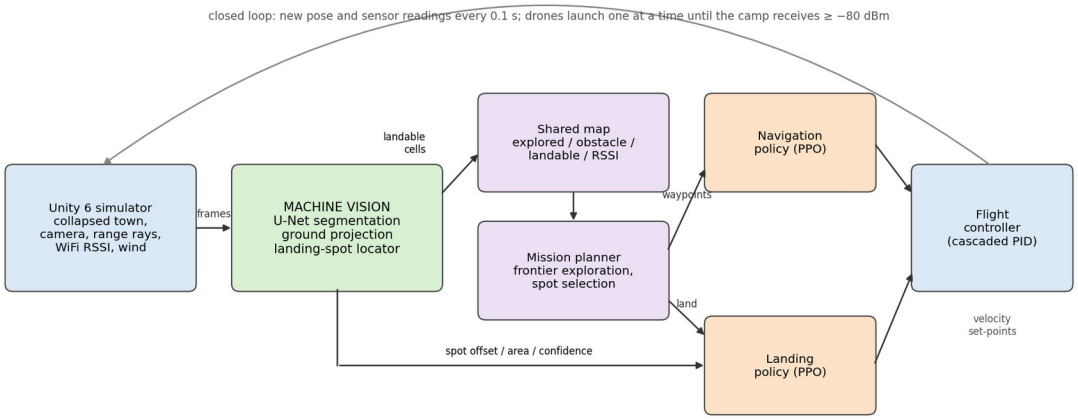


Figure 2. System architecture. The mission planner issues waypoints and landing commands; the PPO navigation and landing policies (orange) output velocity set-points to the flight controller.

6.2 Quadrotor model and flight controller

The learned policies never command motors. Like a companion computer talking to an autopilot, they send SetVelocity(v_forward, v_side, v_up, yaw_rate) in the body-yaw frame, limited to 3 m/s, 1 m/s and 90°/s. Two physics models implement this interface. The full model is a 6-DOF rigid-body quadrotor (1.2 kg, 0.2 m arm, four rotors with 7 N maximum thrust and a 30 ms motor time constant, aerodynamic drag) controlled by a cascade: a velocity PI(D) loop computes the desired acceleration, which is converted into a thrust vector and a tilt set-point ($\leq 30°$), followed by proportional attitude and body-rate loops and an inverted 4×4 mixer that allocates collective thrust and torques to the rotors. It is integrated at 500 Hz. The fast model, used for training, makes the velocity follow the command through a first-order lag ($\tau = 0.26$ s horizontal, 0.21 s vertical) with acceleration limits and a gust disturbance; its parameters were tuned so that its step response and gust rejection match the full model.

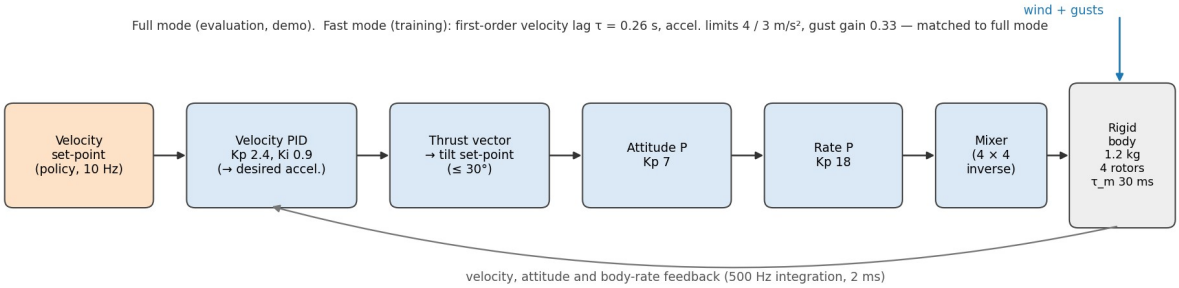


Figure 3. Cascaded flight controller of the full quadrotor model and the matched fast model used for training.

Table 4. Flight-controller parameters

Parameter	Value
Mass, arm length, rotors	1.2 kg, 0.2 m, 4 × 7 N max, motor lag 30 ms, yaw torque coeff. 0.016
Velocity loop	Kp 2.4, Ki 0.9, Kd 0.05 (horizontal); Kp 4.0, Ki 1.5 (vertical); integrator anti-windup
Attitude / rate loops	Kp 7 / Kp 18 (yaw rate Kp 4); tilt limit 30°, horizontal acceleration ≤ 5 m/s²
Fast surrogate	τ = 0.26 / 0.21 / 0.19 s (horizontal / vertical / yaw), accel. limits 4 / 3 m/s², gust gain 0.33
Integration	full 2 ms (5–50 sub-steps per decision), fast 20 ms; decisions every 0.1 s

6.3 Navigation as a Markov decision process

Task: reach a waypoint chosen by the planner, through streets the drone has never seen, at cruise altitude. The policy does not receive the map — that is the reason for learning it instead of planning a path. Observations are egocentric and available on a real drone: distance to the waypoint (log-scaled) and its bearing relative to the heading, body-frame velocity, heading (sin, cos), a ring of 24 range measurements every 15° up to 10 m (e.g. from a 360° LiDAR or ToF array) and the previous action, 33 values in total. Actions are three continuous values in [−1, 1] scaled to forward speed, sideways speed and yaw rate.

Table 5. Navigation reward (per 0.1 s step)

Reward term	Value	Purpose
Progress along the street path to the waypoint	Δ(path distance) / initial path distance (Σ ≈ +1)	dense guidance that rewards going around walls, not into them
Time cost	−0.0005 per step	prefer fast arrival
Wall proximity (< 1 m)	up to −0.02 per step	keep a safety margin

Reward term	Value	Purpose
Action smoothness	$-0.002 \times \lvert\Delta\text{action}\rvert^2$	avoid jerky commands
Collision (episode ends)	−1	safety
Waypoint reached (< 1 m)	+1	task completion

Progress is measured on a shortest-path distance field (Dijkstra on the true grid), which is used only by the reward and never observed by the policy. Using straight-line distance instead would reward flying into the wall between the drone and the waypoint; the path field makes detours and turning back out of dead ends rewarding. Episodes end on collision, success or a step limit.

6.4 Landing as a Markov decision process

Task: descend from cruise altitude onto the chosen patch under gusts and imperfect detections. Observations are ten values from the vision module and onboard sensors — never raw pixels: patch-centre offset in the nadir image (x, y), apparent area, detection confidence (zero when not detected), edge clearance, altitude from a rangefinder, body velocity (3) and the number of frames since the last detection. Actions are horizontal velocity ($\leq$ 1.5 m/s) and vertical velocity ($\leq$ 1 m/s).

Table 6. Landing reward

Reward term	Value
Horizontal offset from patch centre	−0.01 × offset (m) per step
Descending too fast near the ground	−0.05 × excess vertical speed per step
Time cost / action smoothness	−0.002 per step / $-0.002 \times \lvert\Delta\text{action}\rvert^2$
Potential-based shaping	$+ (\Phi_t - \Phi_{t-1})$, $\Phi = -0.1 \cdot (\text{altitude} + 2 \cdot \text{offset})$
Soft touchdown fully inside the patch ($\leq$ 0.6 m/s)	+1 + centring bonus (0…1)
Off patch, hard impact, patch lost > 3 s, fly-away, timeout (40 s)	−1 (episode ends)

The shaping term follows Ng et al. [17]: because it is a difference of a potential, it does not change which policy is optimal, but it rewards every step that makes the drone lower and better centred, which removes the sparse-reward problem of the touchdown bonus.

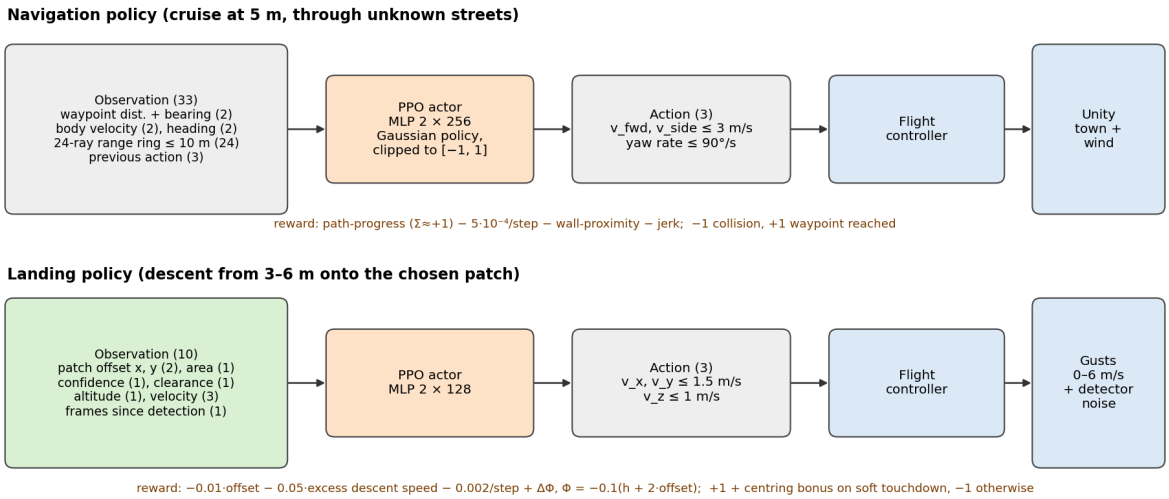


Figure 4. Agent–environment loops of the two PPO policies: observations, network, actions and reward.

6.5 Proximal Policy Optimization

Both policies are trained with PPO [14] in Unity ML-Agents. PPO collects trajectories with the current stochastic policy π_θ (a Gaussian over the continuous actions), estimates advantages A_t with generalised advantage estimation (GAE, $\lambda = 0.95$) [16], and maximises the clipped surrogate objective

$$L(\theta) = E_t\left[\min\left(r_t(\theta)A_t,\ \mathrm{clip}(r_t(\theta), 1-\varepsilon, 1+\varepsilon)A_t\right)\right], \quad r_t(\theta) = \frac{\pi_\theta(a_t \mid s_t)}{\pi_{\theta_{old}}(a_t \mid s_t)},$$

together with a value-function loss and an entropy bonus (β). Clipping the probability ratio to $[1-\varepsilon, 1+\varepsilon]$ keeps each update close to the data-collecting policy, which is what makes PPO stable with large batches from many parallel arenas. Observations are normalised online.

Table 7. PPO configuration (one configuration file per run)

Hyper-parameter	Navigation	Landing
Network (actor and critic)	MLP 2 × 256	MLP 2 × 128
Learning rate (constant) / entropy β	$3\cdot10^{-4}$ / $3\cdot10^{-3}$	$3\cdot10^{-4}$ / $3\cdot10^{-3}$
Batch / buffer size	2,048 / 32,768	1,024 / 20,480
Discount γ / GAE λ / clip ε	0.995 / 0.95 / 0.2	0.99 / 0.95 / 0.2
Epochs per update / time horizon	3 / 256	3 / 128
Curriculum	town level 1 → 4 (Table 1)	gusts 0 → 1.5 → 3 → 4.5 m/s, mean wind 0 → 1.5 m/s
Promotion criterion	smoothed reward ≥ 1.6 / 1.55 / 1.5	smoothed reward ≥ 1.3 / 1.2 / 1.1

6.6 Curriculum and robustness to model error

Both policies are trained with a curriculum [18]: navigation starts in open level-1 towns with short waypoints and is promoted to denser, windier towns with waypoints up to 80 m away; landing starts in calm air and is promoted through three gust levels. Promotion is automatic once the smoothed episode reward passes a threshold. To make the landing policy robust to the real perception system, it is fine-tuned with a detector noise model calibrated from the real U-Net and locator on held-out towns: 7.3 % missed detections, 41.7 % false detections when the patch is out of view, 0.26 (normalised) localisation noise with 0.02 bias, and 2 % dropped frames. This is a form of domain randomisation over the sensing channel [19]. As a project rule, the reward and the curriculum were never changed in the same run, and every run has its own committed configuration file.

6.7 Mission planner

Each drone runs a state machine. It downloads the shared map, transits to its parent relay along explored streets, explores, navigates to the selected spot, hovers to confirm the radio link, lands, and uploads what it has learned. If the camp is covered the mission ends; otherwise the next drone launches. A crashed drone is replaced by the next one, up to six drones; each drone has 900 s of battery and stops exploring after 300 s or at 50 % battery.

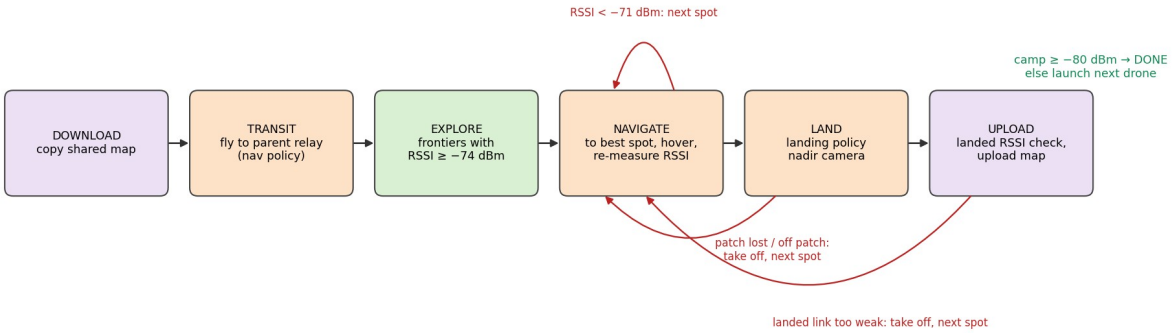


Figure 5. Per-drone mission state machine with the recovery transitions (red).

- **Exploration rule:** among frontier cells (known free cells next to unknown space) whose predicted RSSI to the parent is ≥ −74 dBm, go to the one with the shortest street-path distance to the camp. Predictions through unexplored space carry a 6 dB uncertainty penalty so the drone does not follow optimistic predictions out of radio range.

- **Spot rule:** among cells the shared map believes are landable, keep those with room for the drone (≥ 3 of 4 neighbours landable), RSSI to the parent ≥ −71 dBm (measured if visited, otherwise predicted), at least 5 m of path progress toward the camp, and reachable; choose the one closest to the camp by street path. Spots up to 2 dB below the threshold are visited and re-measured by hovering rather than discarded.
- **Shared map:** explored, obstacle, landable evidence and measured RSSI per cell plus the relay list; each drone works on a copy and uploads only its own observations after landing.
- **Camera gimbal:** 45° forward-down while exploring, aimed at the planner's current goal with a ±60° pan, and straight down while landing; decoupling the camera from the body yaw was necessary because the navigation policy often flies sideways.

6.8 Perception interface (summary)

The vision module is described in detail in the companion machine-vision report. A 0.48 M-parameter U-Net segments each 128×128 camera frame into landable, not landable, obstacle and unknown; the predictions are projected onto the ground with the known camera pose and fused into the shared map; and in the nadir view a connected-component locator produces the patch offset, area, confidence and clearance that form the landing policy's observations. For RL training the camera is not rendered; the calibrated noise model produces the same outputs at negligible cost.

6.9 Baselines

Table 8. Baselines

Task	Baseline	Description
Navigation	Straight line	fly toward the waypoint, no avoidance (lower bound)
Navigation	Potential field	attraction to the waypoint + repulsion from range rays < 1.5 m; same sensors as the policy
Navigation	A* + reactive follower	A* on the true map, then follow the path (upper bound: has the map)
Landing	PID visual servoing	centre over the detected patch with a P controller, descend when centred, climb slowly when the patch is lost
Mission	Scripted stand-ins	potential-field navigation and/or PID landing in place of the policies

Task	Baseline	Description
Mission	Spot rules	random qualifying spot, or the qualifying spot nearest to the parent

7. Implementation

Table 9. Implementation stack

Component	Technology
Simulator	Unity 6 (6000.6.4f1), C#; all scenes, maps and builds generated from code
Reinforcement learning	Unity ML-Agents 4.1 (C#) + mlagents 1.1.0 / PyTorch 2.5 (Python), PPO, CPU training
Policy deployment	ONNX export → Unity Inference Engine 2.6, executed every 0.1 s inside the mission
Vision	U-Net in PyTorch (GPU training), ONNX in Unity (CPU backend, ≈ 4 ms per frame)
Tooling	uv project environment (Python 3.10), Unity MCP for editor inspection, git, one config file per run
Compute	Intel i5-13400F (16 threads), 32 GB RAM, NVIDIA RTX 3060 12 GB

7.1 Code organisation

The Unity code follows the module boundaries of the system: Core (town generator, curriculum, shortest-path field, radio model), Flight (wind model and both flight models behind one interface), RL (NavAgent, LandAgent, baselines, training manager with parallel arenas, evaluation runner), Vision (segmenter, locator, noise model) and Mission (shared map, per-drone state machine, policy runner, mission runner for headless evaluation and the live demo). Python holds the training configurations, evaluation scripts and plotting. Every run is reproducible from its configuration file and seeds.

7.2 Training infrastructure

Rendering is not needed to train either policy: navigation uses the range rays cast on the grid, and landing uses the calibrated detector noise model. Each Unity process therefore runs 32 grid-only arenas headless, giving about 4,500 environment steps per second for navigation; the Python trainer, not the simulator, is the bottleneck. Training ran on the CPU. During parallel training the CPU package reached 100 °C, so a resource monitor logged temperatures every 30 s and paused training processes at ≥ 98 °C; afterwards at

most about three busy cores were used at a time. In total about 2.7 hours of compute were spent on RL and vision training.

7.3 Training runs and reward iteration

Table 10. Training runs (one configuration file per run)

Run	Change	Steps	Final reward	Training success
nav_ppo_v1	from scratch, curriculum level 1 → 4	10.75 M	1.45	83 % (level 4)
nav_ppo_v2	fine-tune on final fast-model parameters	10.0 M	1.46	83 % (level 4)
land_ppo_v1	initial reward (timeout −0.5)	1.46 M	−2.18	0 % — hovered until timeout
land_ppo_v2	reward fix: timeout −1 + potential shaping	4.3 M	2.30	100 %
land_ppo_v3	fine-tune with measured detector noise	+3.7 M	2.16	99.7 %
nav_ppo_v1 (deployed)**	v2 did not beat v1 on held-out towns			

The first landing run is an instructive reward-design failure. Timing out cost −0.5 while a failed landing cost −1, so in early training — when most landing attempts failed — hovering until the timeout was the highest-value behaviour, and the policy converged to never landing (episode length stuck at the 400-step limit, see the landing training curves below). Making the timeout as costly as a failure and adding the potential-based shaping term fixed this: v2 reached 100 % success and the hardest gust lesson in under one million steps. The navigation fine-tune (v2) did not improve held-out success, so v1 is deployed.

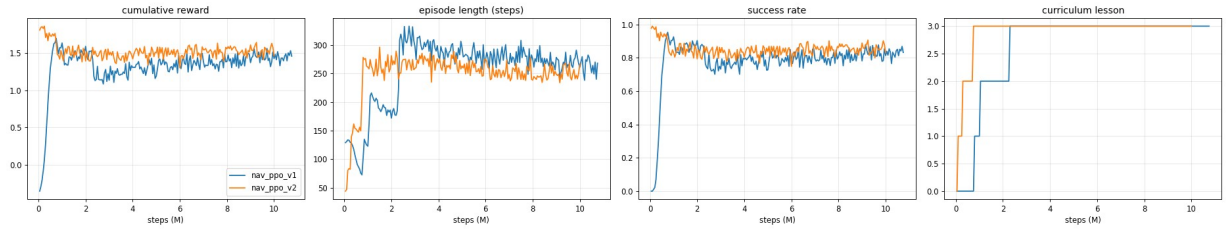


Figure 6. Navigation training: cumulative reward, episode length, success rate and curriculum lesson (0 = level 1 … 3 = level 4). Reward and success drop at each promotion and recover.

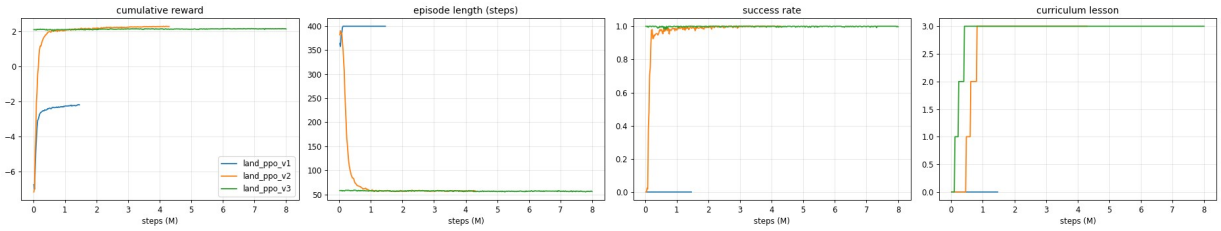


Figure 7. Landing training: v1 converges to hovering (reward ≈ −2, episode length at the 400-step limit); v2 with the reward fix lands in ≈ 60 steps; v3 continues from v2 with the measured detector noise.

7.4 Deployment inside the mission and live demonstration

The trained policies are exported to ONNX and executed by the Unity Inference Engine inside the mission loop at 10 Hz with the same observation code used in training, so the deployed path is identical to the trained one. A live demonstration runs in the editor or as a standalone build with six maps (four hand-built layouts — Z-route, U-turn, Staircase and Blocked boulevard — and two held-out procedural towns). The display shows the mission mode, signal to the parent, weakest link, battery, detection confidence, step reward and wind, a minimap of the shared map, the camera view with live segmentation, and radio links coloured by strength.

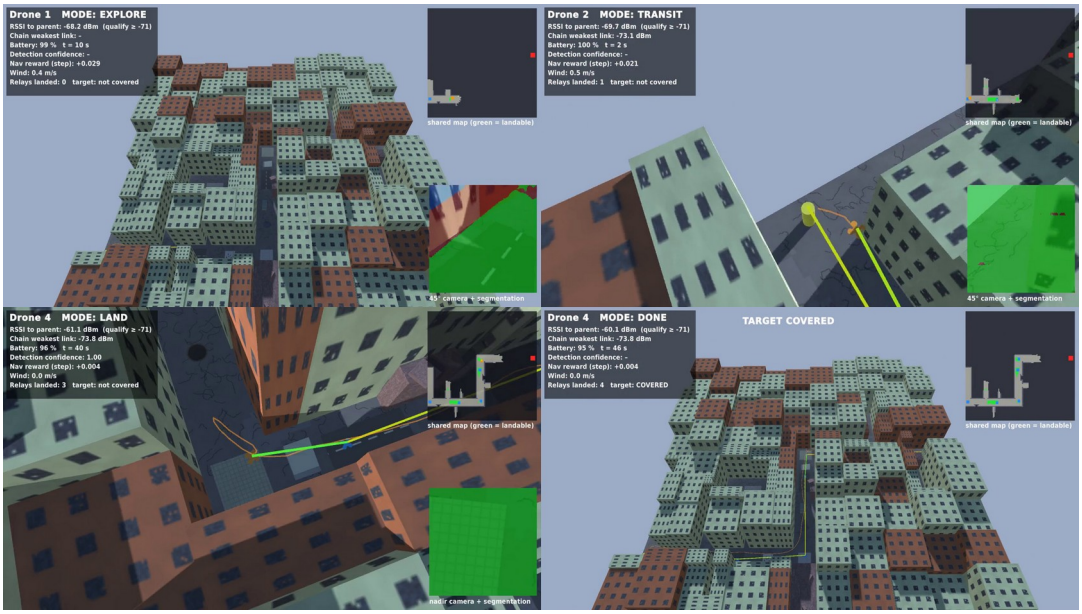


Figure 8. Demonstration: exploring with the 45° camera (top left), navigating to a selected spot while keeping −71 dBm to the parent (top right), learned landing with the nadir camera (bottom left), and the completed four-relay chain covering the camp (bottom right).

8. Experimental Results

All results are on held-out (odd-seed) towns. Policy evaluations use 100 episodes per condition; mission evaluations use 100 missions per condition with up to six drones.

8.1 Flight-controller validation

Table 11. Step responses of the full quadrotor and the fast training model

Axis	Model	Rise 10–90 % (s)	Overshoot	Settling 5 % (s)	Max tilt
v_x (0 → 2 m/s)	Full	0.54	1.0 %	0.83	26.8°
v_x (0 → 2 m/s)	Fast	0.61	0 %	0.84	–
v_z (0 → 0.8 m/s)	Full	0.41	1.8 %	0.55	–
v_z (0 → 0.8 m/s)	Fast	0.45	0 %	0.62	–
yaw (0 → 60°/s)	Full	0.42	0 %	0.62	–
yaw (0 → 60°/s)	Fast	0.41	0 %	0.56	–

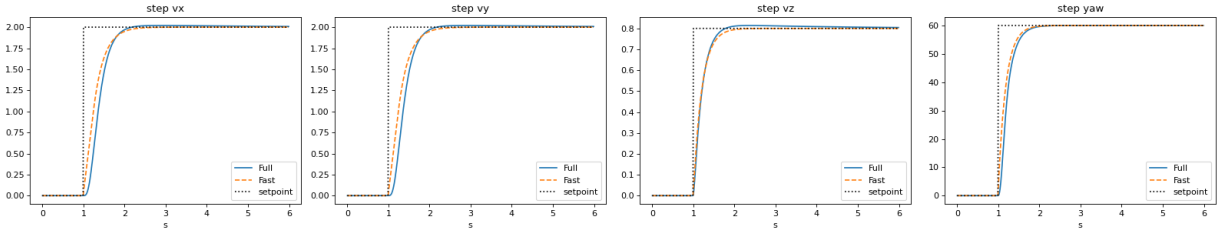


Figure 9. Velocity and yaw-rate step responses of the full quadrotor and the matched fast model.

Under a 3 m/s gust both models drift at most 0.38 m from a held position (RMS 0.12–0.13 m), and on a 20 m square at 2 m/s the cross-track RMS error is 0.066 m (full) and 0.067 m (fast). The surrogate therefore reproduces the closed-loop behaviour that matters to the policies, at a fraction of the cost.

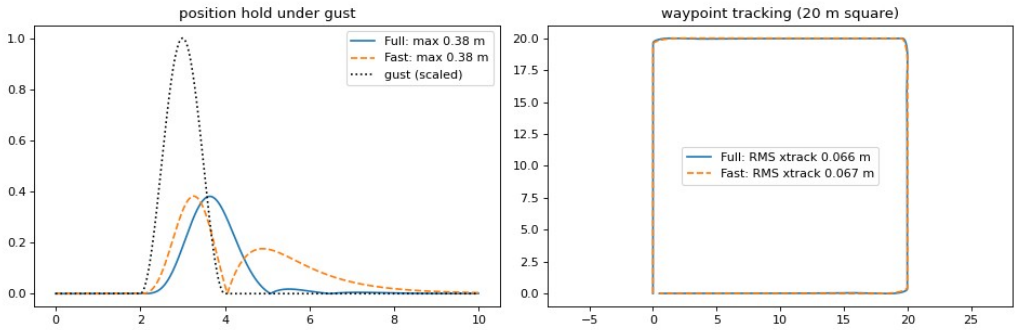


Figure 10. Position hold under a 3 m/s gust (left) and waypoint tracking on a 20 m square (right).

8.2 Navigation

Table 12. Navigation success on held-out towns (100 episodes per level, fast flight model)

Controller	Level 1	Level 2	Level 3	Level 4
Straight line	84 %	64 %	39 %	18 %
Potential field	93 %	92 %	78 %	54 %
PPO policy (ours)	**98 %**	**98 %**	**93 %**	**80 %**
A* with full map (upper bound)	100 %	100 %	99 %	94 %

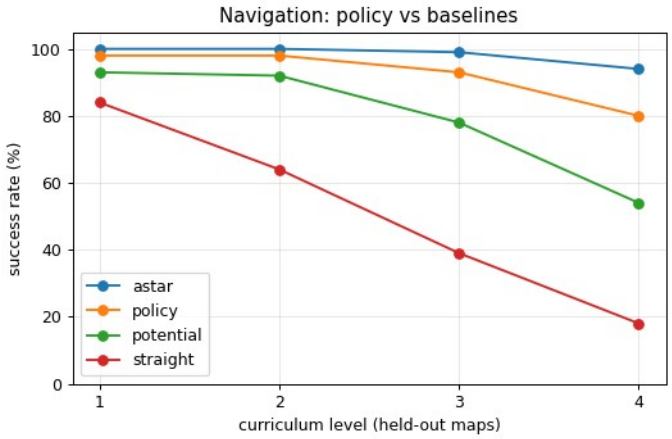


Figure 11. Navigation success versus town difficulty.

Table 13. Failure modes and transfer to the full quadrotor

Controller / level	Success	Collision	Timeout	Path ÷ optimal	Time to waypoint (s)
PPO, level 2, fast	98 %	1 %	1 %	1.07	8.0
PPO, level 2, full quadrotor	95 %	3 %	2 %	1.16	8.9
PPO, level 4, fast	80 %	4 %	16 %	1.19	24.0
PPO, level 4, full quadrotor	74 %	8 %	18 %	1.17	23.4
Potential field, level 4	54 %	0 %	46 %	0.99	21.7
Straight line, level 4	18 %	82 %	0 %	0.95	13.9

Without a map, the learned policy closes most of the gap between reactive control and planning with full knowledge: on the hardest towns it succeeds 80 % of the time versus

54 % for the potential field with identical sensors. The potential field rarely collides but gets trapped in local minima in front of rubble and in dead ends (46 % timeouts); the policy has learned to turn back and search along the street. Its paths are 7–19 % longer than the shortest path because it explores. Transferring the policy from the fast training model to the full quadrotor costs 3–6 percentage points, mostly as extra collisions in narrow streets, which confirms that matching the surrogate to the full dynamics was worthwhile.

8.3 Landing

Table 14. Landing success and touchdown error with the measured detector noise (100 episodes each)

Gust amplitude (mean wind)	PPO landing (v3)	PID visual servoing
0 m/s (0)	100 %, error 0.01 m	97 %, 0.02 m
1.5 m/s (0.5)	100 %, 0.02 m	91 %, 0.04 m
3.0 m/s (1.0)	100 %, 0.02 m	76 %, 0.07 m
4.5 m/s (1.5)	100 %, 0.03 m	64 %, 0.11 m
6.0 m/s (2.0) — beyond training	100 %, 0.05 m	44 %, 0.14 m
3.0 m/s, 2 × detector noise	100 %, 0.08 m	0 %
3.0 m/s, full quadrotor	96 % (3 % hard impact)	33 %

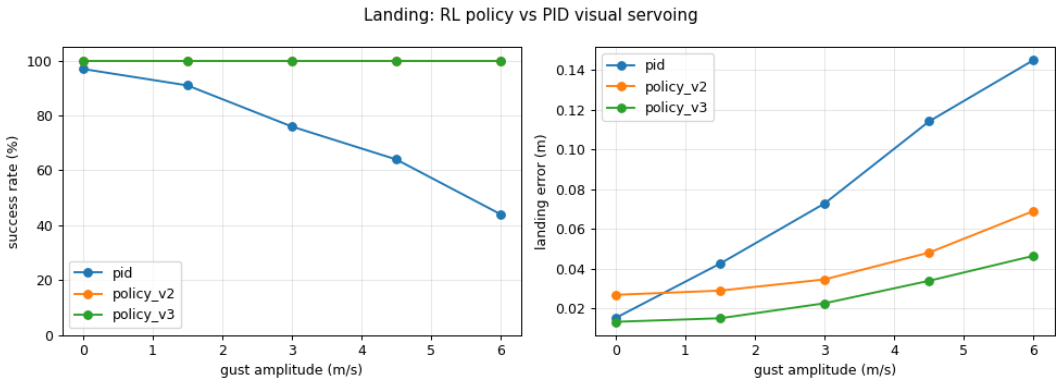


Figure 12. Landing success (left) and touchdown error (right) versus gust amplitude.

All PID failures are lost-patch timeouts: a gust or a false detection pulls the controller off the patch, the patch leaves the narrow nadir field of view, and the controller cannot reacquire it. The learned policy leans into gusts, keeps the patch in view, and keeps descending when the detection is temporarily unreliable instead of chasing it. It generalises to 6 m/s gusts, beyond the 4.5 m/s of its curriculum, and fine-tuning with the measured noise (v3) halved the touchdown error compared with v2. Its weakness is touchdown speed: it lands at about 0.5 m/s, close to the 0.6 m/s limit, so on the full quadrotor a few landings exceed the limit; a stricter speed penalty near the ground is the obvious fix.

8.4 Mission-level results

Table 15. Final system: 100 held-out missions each, up to six drones

Final system, level-2 towns	Camp covered	Range extended toward camp (m)
Perfect perception (ground truth)	51 %	70.3 ± 34.0
Real U-Net detector in the loop	**45 %**	**62.3 ± 35.0**

A mission lands about three relays on average, each advancing the chain about 18 m along the street path, and the weakest link of a completed chain is about −72.5 dBm, comfortably above the −80 dBm threshold. To isolate the contribution of each autonomy component, the following ablations replace one component at a time (100 missions each, level 2, calibrated detector noise so that only the component under test changes). The ablations were run with the planner version before the demonstration fixes of Section 9.4, so their absolute numbers differ slightly from the final-system table above.

Table 16. Mission ablations (level 2, 100 held-out missions each)

Condition	Camp covered	Range (m)	Relays landed	Drones lost / mission
Full system (PPO navigation + PPO landing)	**47 %**	**64.4**	3.1	0.28
Scripted landing (PID visual servoing)	42 %	61.8	3.0	0.27
Scripted navigation (potential field)	9 %	35.0	1.7	2.60
Full quadrotor instead of fast model	30 %	53.9	2.6	1.36
Spot rule: random qualifying spot	25 %	52.3	3.9	0.24
Spot rule: nearest to parent	3 %	33.5	4.2	0.27
Shared map off (drones 2…k)	42 %	62.6		

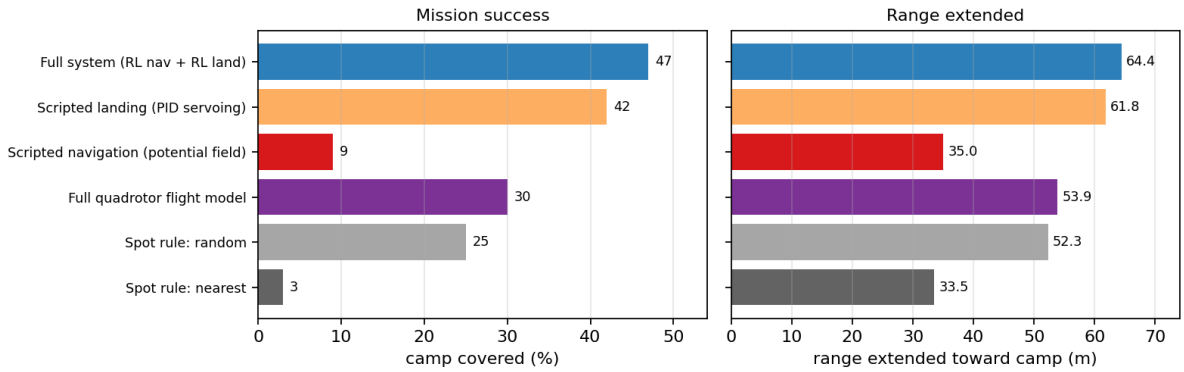


Figure 13. Contribution of each component to mission success and range.

- **Learned navigation is essential:** with the potential-field controller drones get stuck or crash in dead ends (2.6 drones lost per mission) and only 9 % of missions reach the camp.
- **Learned landing helps but less:** the mission lands from cruise altitude in mild level-2 wind, where the PID baseline still mostly succeeds; the gap grows with wind (Section 8.3).
- **Flight-model fidelity matters at mission scale:** small per-flight differences (3–6 points) compound over many flights per mission and reduce success from 47 % to 30 %; training on or fine-tuning with the full model is the next step.
- **Where to land matters as much as how:** the radio-aware spot rule (furthest progress that still hears the parent) beats random spots by 22 points and the nearest spot by 44 points.
- **The shared map saves battery:** later drones explore 30 s instead of 52 s and use 68 s instead of 92 s of battery, because they inherit what earlier drones learned.

Table 17. Mission success versus town difficulty (calibrated detector noise)

Town difficulty	Camp covered	Range extended (m)
Level 2	47 %	64.4 ± 35.8
Level 3	17 %	52.8 ± 42.6
Level 4	2 %	32.8 ± 32.6

Mission success drops sharply on harder towns. Failure dumps show that this is mostly geometric: in level-3 and level-4 towns landable surfaces are scarce, and if no landable spot exists at a street corner within about 35 m of the parent relay, the chain cannot turn that corner. This is a property of the scenario rather than of flight control, and is the main motivation for the future-work items in Section 11.

8.5 Effect of perception quality on the mission

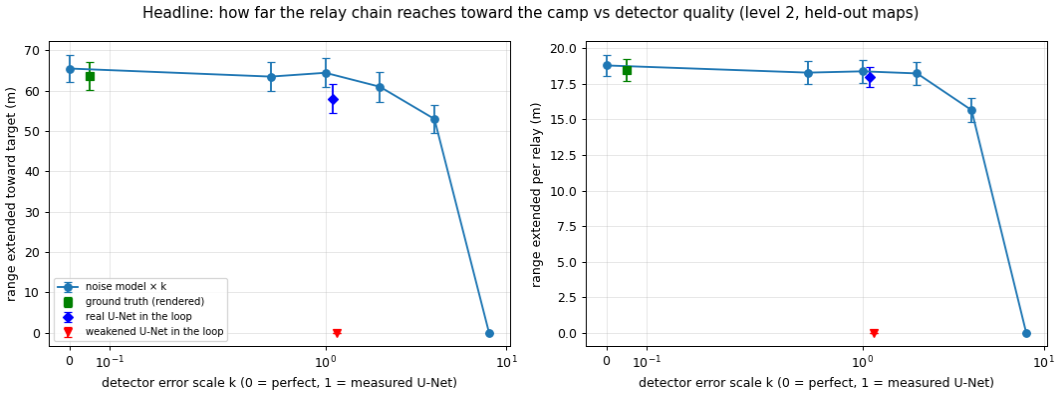


Figure 14. Range of the relay chain as the detector's error rates are scaled by k (k = 0 perfect, k = 1 measured U-Net, k > 1 degraded).

The landing policy and planner tolerate up to about twice the measured detector error with no loss of range; at four times the error the range drops by 18 %, and at eight times no relay lands. Training the landing policy on the measured noise is what gives this margin.

8.6 Live-demonstration reliability

Table 18. Demonstration maps over ten wind / noise / lighting seeds each

Demonstration map	Before fixes	After fixes
Z-route	5/10	**10/10**
U-turn	9/10	**10/10**
Staircase	9/10	**10/10**
Blocked boulevard	5/10	**8/10**
Procedural town A (replaced by another level-2 town)	5/10	**10/10**
Procedural town B (level 3, replaced by a level-2 town)	1/10	**10/10**
Total	**34/60**	**58/60**

The fixes are described in Section 9.4. After a fixed shadow-map resolution was introduced, the live demo and the headless evaluation produce identical trajectories for the same seed.

9. Performance Evaluation

9.1 Comparison with the DA-1 proposal

Table 19. Objectives versus outcomes

DA-1 objective / plan	Outcome
Autonomous relay position selection	Radio-aware frontier exploration and spot selection; each relay lands ≈ 18 m further along the street path toward the camp
Improved coverage and signal strength	Real-detector chain covers the camp in 45 % of held-out level-2 missions and extends coverage by 62 m on average; weakest link ≈ −72.5 dBm
PPO-based decision making	Two PPO policies: navigation 98/98/93/80 % (levels 1–4); landing 100 % at gusts up to 6 m/s
Pixhawk-style flight-controller interface	Policies output velocity set-points to a cascaded PID autopilot model, as in PX4 offboard mode
Single drone repositioning (future: multiple UAVs)	Extended to a chain of up to six perching drones with a shared map
Simulation first (AirSim / Gazebo / PX4 SITL)	Unity ML-Agents simulator with a validated quadrotor model; hardware and SITL are future work
FastDepth encoder features as RL state	Replaced by a range-sensor ring (navigation) and U-Net landing-spot features (landing): smaller, more direct inputs

9.2 Strengths

- Learned navigation clearly beats reactive control with the same sensors (80 % vs 54 % on the hardest towns) and makes multi-drone missions possible at all (47 % vs 9 %).
- Learned landing is robust to gusts beyond its training range and to twice the measured detector noise, where PID visual servoing fails completely.
- The policies use an autopilot-style velocity interface and onboard-available observations, so they can be placed on top of a PX4 flight stack.
- Every claim is measured on held-out towns, with baselines and ablations of every component, and every training run has its own configuration file.

9.3 Limitations

- Simulation only: the radio model is a signal-strength proxy (no multipath, interference or throughput) and the quadrotor model omits ground effect, battery voltage sag and sensor latency.
- Policies were trained on the fast surrogate and lose 3–6 points on the full quadrotor per flight, which compounds to 47 % → 30 % at mission level.

- Flight is 2.5-D: a fixed 5 m cruise altitude and vertical motion only for landing; flying over low obstacles is not exploited.
- Mission success on dense towns is limited by the availability of landable spots at corners (level 4: 2 %).
- The landing policy touches down at ≈ 0.5 m/s, close to the 0.6 m/s limit.

9.4 Failure analysis of the live demonstration

Two demonstration maps failed live although they had succeeded in headless tests. Every demonstration map was re-run over ten wind, noise and lighting seeds and each failure was classified from logged map dumps. The causes were mostly at the interface between the flight policy and the rest of the system:

- **Body yaw versus sensing direction:** the navigation policy controls yaw freely and was trained only to reach waypoints, so it often flew the last stretch sideways or backwards (headings 223–275° when approaching a corner). A body-fixed camera therefore never saw the corner where the next relay had to land. Aiming the gimbal at the planner's goal, independent of the body yaw, raised the worst map from 1/10 to 10/10 and the demo total from 30/60 to 43/60.
- **Optimistic radio prediction:** a drone followed a predicted signal into an unexplored dead end and flew out of radio range; adding a 6 dB uncertainty penalty for unexplored stretches stopped this.
- **Hard thresholds on noisy measurements:** a corner spot that read −72.2 dBm against the −71 dBm requirement was discarded on a single noisy reading; spots within 2 dB are now visited and re-measured by hovering.
- **Non-determinism:** the renderer sized the shadow map from the window resolution, so the windowed demo and headless evaluation diverged after a few drones; a fixed resolution made them identical.

10. Conclusion

This project delivered a simulation-validated autonomous relay-drone system in which drones fly out one by one, navigate through unknown collapsed streets, land on safe surfaces and form a radio relay chain toward a disconnected camp. A validated cascaded-PID quadrotor model with a matched fast surrogate made it possible to train the two deployed PPO policies in about 80 minutes of CPU time. The navigation policy reaches waypoints in 80–98 % of episodes depending on town difficulty, well above a potential-field controller with the same sensors, and the landing policy, trained with potential-based shaping, a gust curriculum and the measured detector noise, lands within a few centimetres in 100 % of episodes at gusts up to 6 m/s while PID visual

servoing degrades to 44 %. With the real detector in the loop, the complete system covered the camp in 45 % of 100 held-out level-2 missions, extending coverage by 62 m on average.

Mission-level ablations show where autonomy matters: learned navigation raises mission success from 9 % to 47 %, the radio-aware landing rule is as important as the flight skills, and sharing a map between drones cuts exploration time by 40 %. The failure analysis showed that integrating a learned policy into a larger system requires care at the interfaces — sensing direction, uncertain measurements and determinism — as much as policy accuracy itself.

11. Future Scope

- **PX4 software- and hardware-in-the-loop:** run the exported policies as an offboard node sending MAVLink velocity set-points to PX4 SITL in Gazebo, then on the Pixhawk and RV1103 companion computer proposed in DA-1.
- **Closing the dynamics gap:** fine-tune the policies on the full quadrotor model and randomise mass, motor constants, drag and latency during training.
- **Better landing behaviour:** a stricter touchdown-speed penalty, a final flare manoeuvre and landing on slightly sloped surfaces.
- **Richer radio and energy models:** ray-traced or measured propagation, throughput and packet loss as in DA-1, and battery-aware decisions (re-launching depleted relays).
- **Multi-agent RL and 3-D flight:** let relays re-position after landing, learn exploration and spot selection jointly, and use altitude changes to fly over rubble.
- **Real-world validation:** short flights with a single relay drone to calibrate the radio and wind models before chain experiments.

12. References

[1] FSO-Based Drone Assisted Mobile Access Network for Emergency Communications (surveyed in DA-1).

[2] Multi-UAV Wireless Mesh Networks for Disaster Recovery, 2026 (surveyed in DA-1).

[3] Blockchain-Enhanced UAV Networks for Post-Disaster Communication, 2026 (surveyed in DA-1).

[4] M. Mozaffari, W. Saad, M. Bennis, Y.-H. Nam and M. Debbah, "A Tutorial on UAVs for Wireless Networks: Applications, Challenges, and Open Problems," IEEE Communications Surveys & Tutorials, vol. 21, no. 3, 2019.

[5] A. Al-Hourani, S. Kandeepan and S. Lardner, "Optimal LAP Altitude for Maximum Coverage," IEEE Wireless Communications Letters, vol. 3, no. 6, 2014.

[6] R. Mahony, V. Kumar and P. Corke, "Multirotor Aerial Vehicles: Modeling, Estimation, and Control of Quadrotor," IEEE Robotics & Automation Magazine, vol. 19, no. 3, 2012.

[7] L. Meier, D. Honegger and M. Pollefeys, "PX4: A Node-Based Multithreaded Open Source Robotics Framework for Deeply Embedded Platforms," in Proc. IEEE ICRA, 2015.

[8] J. Hwangbo, I. Sa, R. Siegwart and M. Hutter, "Control of a Quadrotor with Reinforcement Learning," IEEE Robotics and Automation Letters, vol. 2, no. 4, 2017.

[9] W. Koch, R. Mancuso, R. West and A. Bestavros, "Reinforcement Learning for UAV Attitude Control," ACM Transactions on Cyber-Physical Systems, vol. 3, no. 2, 2019.

[10] E. Kaufmann, L. Bauersfeld, A. Loquercio, M. Müller, V. Koltun and D. Scaramuzza, "Champion-level drone racing using deep reinforcement learning," Nature, vol. 620, 2023.

[11] R. Polvara, M. Patacchiola, S. Sharma, J. Wan, A. Manning, R. Sutton and A. Cangelosi, "Toward End-to-End Control for UAV Autonomous Landing via Deep Reinforcement Learning," in Proc. ICUAS, 2018.

[12] A. Rodriguez-Ramos, C. Sampedro, H. Bavle, P. de la Puente and P. Campoy, "A Deep Reinforcement Learning Strategy for UAV Autonomous Landing on a Moving Platform," Journal of Intelligent & Robotic Systems, vol. 93, 2019.

[13] V. Mnih et al., "Human-level control through deep reinforcement learning," Nature, vol. 518, 2015.

[14] J. Schulman, F. Wolski, P. Dhariwal, A. Radford and O. Klimov, "Proximal Policy Optimization Algorithms," arXiv:1707.06347, 2017.

[15] T. Haarnoja, A. Zhou, P. Abbeel and S. Levine, "Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor," in Proc. ICML, 2018.

[16] J. Schulman, P. Moritz, S. Levine, M. Jordan and P. Abbeel, "High-Dimensional Continuous Control Using Generalized Advantage Estimation," in Proc. ICLR, 2016.

[17] A. Y. Ng, D. Harada and S. Russell, "Policy Invariance under Reward Transformations: Theory and Application to Reward Shaping," in Proc. ICML, 1999.

[18] Y. Bengio, J. Louradour, R. Collobert and J. Weston, "Curriculum Learning," in Proc. ICML, 2009.

[19] X. B. Peng, M. Andrychowicz, W. Zaremba and P. Abbeel, "Sim-to-Real Transfer of Robotic Control with Dynamics Randomization," in Proc. IEEE ICRA, 2018.

[20] P. E. Hart, N. J. Nilsson and B. Raphael, "A Formal Basis for the Heuristic Determination of Minimum Cost Paths," IEEE Transactions on Systems Science and Cybernetics, vol. 4, no. 2, 1968.

[21] O. Khatib, "Real-Time Obstacle Avoidance for Manipulators and Mobile Robots," The International Journal of Robotics Research, vol. 5, no. 1, 1986.

[22] B. Yamauchi, "A Frontier-Based Approach for Autonomous Exploration," in Proc. IEEE CIRA, 1997.

[23] A. Elfes, "Using Occupancy Grids for Mobile Robot Perception and Navigation," Computer, vol. 22, no. 6, 1989.

[24] S. Shah, D. Dey, C. Lovett and A. Kapoor, "AirSim: High-Fidelity Visual and Physical Simulation for Autonomous Vehicles," in Field and Service Robotics, 2017.

[25] A. Juliani et al., "Unity: A General Platform for Intelligent Agents," arXiv:1809.02627, 2018.

[26] T. S. Rappaport, Wireless Communications: Principles and Practice, 2nd ed., Prentice Hall, 2002.

[27] ITU-R Recommendation P.2040, "Effects of building materials and structures on radiowave propagation above about 100 MHz."